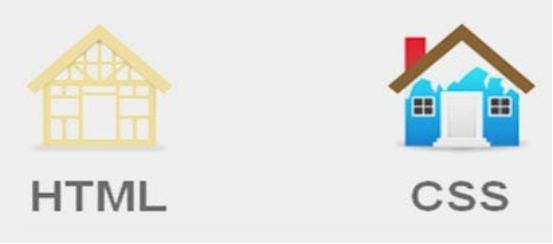
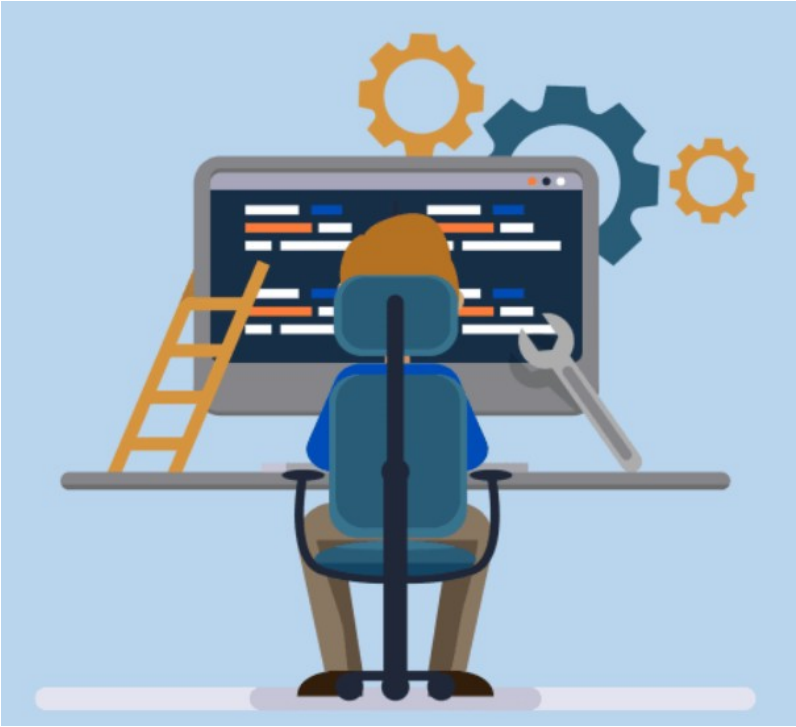
UT2.2 Lenguajes para visualización de la información: CSS
Curso 2024/2025 LENGUAJE DE MARCAS 1º DAW



UT2.2 Lenguajes para visualización de la información: CSS
ÍNDICE



1.¿Qué es CSS? Conceptos Básicos

2. Aplicación de CSS

3. Comentarios

4. Selectores

5.Cajas o Contenedores en CSS

6. Layouts con CSS

7.Enlaces de interes

1. ¿Qué es CSS? Introducción a las Hojas de Estilos

En los entornos científicos en que la Web fue concebida, la gente estaba más preocupada por el contenido de sus páginas que por su presentación.A medida que la Web era descubierta por un espectro mayor de personas de distintas procedencias, la presentación cobró una mayor relevancia,dejando patente las limitaciones de HTML en el ambito estilístico o de presentación.

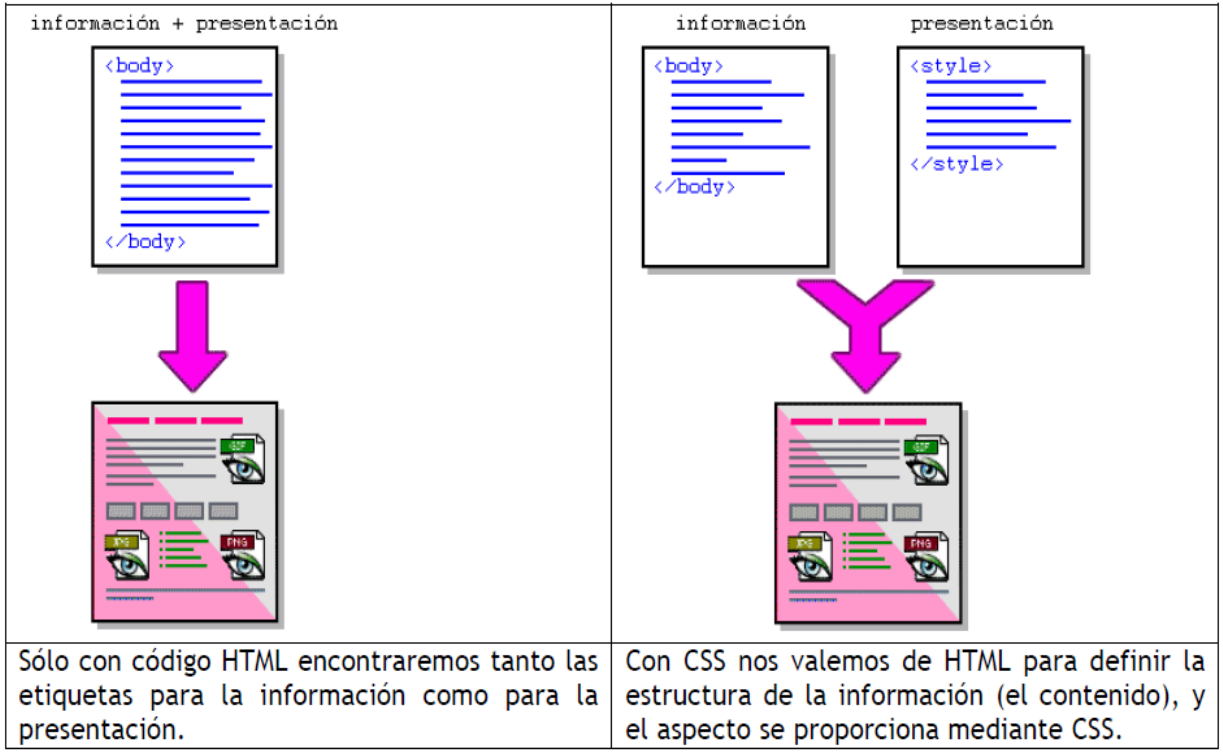
Con las hojas de estilo es más fácil especificar la cantidad de espacio entre líneas, el sangrado de las líneas, los colores a utilizar para el texto y el fondo, el tamaño y estilo de las fuentes, y otros muchos detalles.

Las hojas de estilo representan, por tanto, un avance importante para los diseñadores de páginas web, al darles un mayor rango de posibilidades para mejorar la apariencia de sus páginas. **Con las hojas de estilo definimos las características comunes a todas nuestras páginas una sola vez**, lo que facilita mucho nuestro trabajo cuando tenemos muchas páginas en el servidor.

UT3 Lenguajes para visualización de la información: CSS
1. ¿Qué es CSS? Introducción a las Hojas de Estilos

Las Hojas de Estilo o CSS (Cascading Style Sheets - Hojas de Estilo en Cascada) son un mecanismo que permite aplicar formato a los documentos escritos en HTML (y en otros lenguajes estructurados, como XML) separando el contenido de las páginas de su apariencia:

- la información estará contenida en la página HTML, pero este archivo no debe definir cómo será visualizada esa información
- las indicaciones acerca de la composición visual del documento estarán especificadas en el archivo de la CSS.



Extructura y Sintáxis CSS

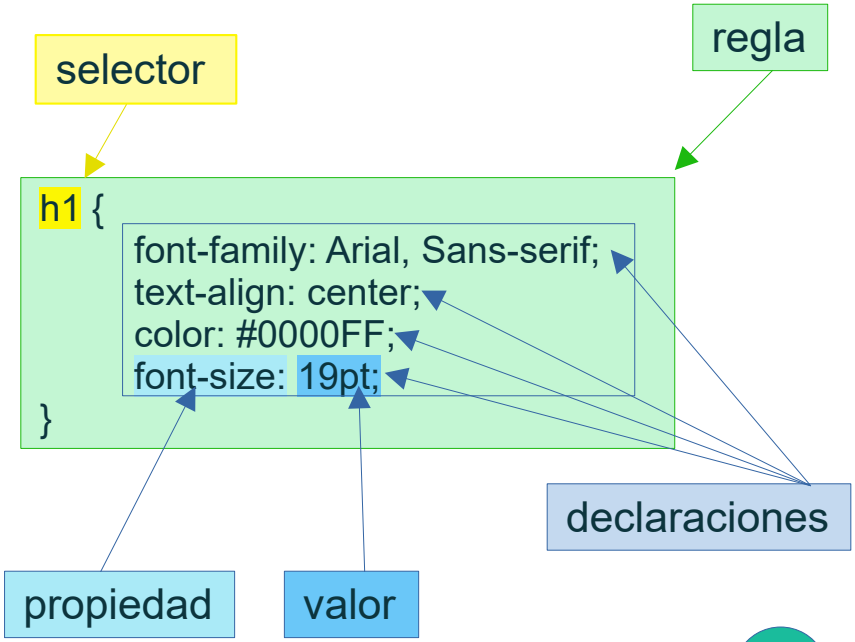
Ejemplo 1 de código CSS: en el especificamos que los encabezados h1 deben mostrarse con tipografía Arial, de 19 puntos, en color azul y alineación central:

El archivo css contiene **reglas** que *constan de un* selector (en este ejemplo, h1) *y una o más* declaraciones (en el ejemplo tenemos cuatro declaraciones).

Cada declaración tiene dos partes: una propiedad (por ejemplo, font-size) y un valor (en este caso, 19pt).

Estas reglas son las que determinan cómo deberá mostrarse la página.

Como se ve en el ejemplo anterior, puede haber varias declaraciones, debiendo separar cada una de la siguiente con el carácter punto y coma (;). Se puede dejar el ; después de la última declaración, pero no es obligatorio.



1. ¿Qué es CSS? Introducción

Como ya sabemos, HTML es un lenguaje de marcado que se usa para establecer la estructura de un sitio en internet. Y con CSS podemos estilizar cada uno de los elementos de nuestro HTML.

Es decir, CSS son las hojas de estilo en cascada que especifican como los documentos HTML se presentan a los usuarios.

Para aplicar estilos a un elemento en HTML hacemos uso de las reglas CSS. Estas se defienden usando un selector y asignado los estilos que se aplicaran a dicho selector.

CSS maneja diferentes tipos de selectores, por ejemplo: ID, clase, etiqueta, entre otros.

Cada regla CSS indica uno o varios estilos que serán aplicados por el navegador a los objetos HTML al cargarse la página.

Cosas a considerar sobre CSS básico para HTML. Un elemento HTML puede tener asignadas uno o varias reglas CSS, sin embargo, los estilos podrían sobrescribirse dejando solo un estilo aplicado a dicho elemento. Es por esta característica que se llaman hojas de estilo en cascada.

2. Aplicación de CSS

Cuando un navegador lee una hoja de estilo, formateará el documento HTML de acuerdo con la información de la hoja de estilo.

Para que las reglas CSS tengan efecto sobre los elementos de un documento HTML deben vincularse con este. Hay tres formas de insertar una hoja de estilo

1) CSS externo:
2) CSS interno
3) CSS en línea

2.1) CSS Externo

Ejemplo 1 CSS y HTML Los estilos externos se definen dentro del elemento <link>, dentro de la sección <head> de una página HTML. Luego, en cada página HTML de nuestro sitio agregamos la etiqueta link indicando el fichero que contiene nuestra hoja de estilo (el archivo con extensión .css) en nuestro caso estilos1.css.

estilos1.css.

```
h1 {
        font-family: Arial, Sans-serif;
        text-align: center;
        color: #0000FF;
        font-size: 19pt;
}
```

Página Web

sin CSS

CSS Externo

A el encabezado h1 de la página actual le hemos aplicado CSS externo aparecerá con fuente arial de 19 puntos, en color azul y alineación central.

Este párrafo es normal

con CSS

CSS Externo

A el encabezado h1 de la página actual le hemos aplicado CSS externo aparecerá con fuente arial de 19 puntos, en color azul y alineación central.

Este párrafo es normal

index.html

```
<!DOCTYPE html>
<html lang="en">

<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>CSS Externo</title>
   <link rel="stylesheet" href="estilos1.css" type="text/css">
</head>

<body>
   <h1>CSS Externo</h1>
   A el encabezado h1 de la página actual le hemos aplicado CSS externo aparecerá con fuente arial de 19 puntos,
      en color azul y alineación central.</h1>
   <p>Este párrafo es normal</p>

</body>

</html>
```

2.1) CSS Externo

El elemento link aparece siempre *en la sección head* del documento y especifica:

```
<link rel="stylesheet" href="estilos1.css" type="text/css">
```

- el tipo de vínculo: a una hoja de estilo ("stylesheet")
- la ubicación de la hoja de estilo a través del atributo "href"
- el tipo de hoja de estilo que se vincula: "text/css"

Ahora, todos los encabezados h1 de las páginas que contienen la referencia al archivo de la CSS tendrán el aspecto que hemos definido.

El documento que contiene todas las definiciones de estilo tendrá extensión .css, y no podrá contener ninguna etiqueta HTML; sólo podrá contener los estilos.

Se pueden importar tantos documentos de estilos como se deseen, siempre teniendo en cuenta que si existen definiciones que entran en conflicto entre los diferentes documentos, el navegador tendrá en cuenta únicamente la última opción leída.

Este método es el más recomendable y el que permite mayor flexibilidad ya que los estilos pueden cambiarse sin tocar el código HTML y la CSS puede ser compartida por varias páginas

2.2) CSS Interno o Incrustado (hojas de estilo incrustadas)

Podemos poner la hoja de estilo dentro de una página HTML *usando el elemento style* en el interior del documento al que se le quiere dar estilo, *en la sección <head>.* De esta forma los estilos serán reconocidos antes de que la página se cargue por completo.

Este método permite aplicar la hoja de estilo sólo a la página que la contiene.Si bien no es tan práctico como usar una CSS externa, **resulta útil cuando en nuestro sitio tenemos algunas pocas páginas que usan un formato distinto al resto**.

Por las **leyes de cascada** de las CSS, en caso de existir una misma regla (una en la hoja externa, otra en la hoja incrustada) con distintos valores, tiene preponderancia la definida en último lugar.

CSS Interno o Inscrustado

A el encabezado h1 de la página actual le hemos aplicado CSS Interno aparecerá en color azul.

Este párrafo tiene aplicado color rojo con CSS Interno

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>CSS Interno</title>
   <style>
      h1{color: blue;}
      p{color: red;}
   </style>
</head>
<body>
   <h1>CSS Externo</h1>
   A el encabezado h1 de la página actual le hemos aplicado CSS Interno aparecerá
      en color azul.</h1>
   <p>Este párrafo tiene aplicado color rojo con CSS Interno</p>
</body>
</html>
```

En muchos casos convendrá usar ambos métodos simultáneamente: poner un elemento link a una hoja externa para aplicar un estilo general y luego crear una hoja incrustada en la que solamente deberemos definir las reglas específicas para esa página.

2.3) CSS en línea

Por último, también es posible **aplicar el estilo directamente en la etiqueta o elemento HTML** utilizando el siguiente formato:

```
<elemento style = "propiedad1: valor1; propiedad2: valor2; ...">
...
</elemento>
```

Nótese que **las declaraciones van entrecomilladas**.
Por ejemplo:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>CSS En Línea</title>
</head>
<body>
   <h1 style="color: white; background-color: darkblue; text-align: center;">CSS En
Línea</h1>
   A el encabezado h1 de la página actual le hemos aplicado CSS en Línea aparecerá
   en fondo color azul oscuto, texto en color blanco y alineado en el centro.</h1>
   <p style="color: red;">Este párrafo tiene aplicado color rojo con CSS en Línea</p>
</body>
</html>
```

Un estilo en línea pierde muchas de las ventajas de una hoja de estilo (al mezclar contenido con presentación). Use este método con moderación.

2.4 Aplicación de CSS. Múltiples Hojas de Estilo. Orden en Cascada

Múltiples hojas de Estilo

Si se han definido algunas propiedades para el mismo selector (elemento) en diferentes hojas de estilo, Se utilizará el valor de la última hoja de estilo leída.

Orden en cascada

¿Qué estilo se usará cuando haya más de un estilo especificado para un elemento HTML?

Todos los estilos de una página se convertirán en una nueva "cascada" hoja por las siguientes reglas, donde el número uno tiene la prioridad más alta:

1. Estilo en línea (dentro de un elemento HTML)
2. Hojas de estilo internas (en la sección principal)
3. Hojas de estilo externas
4. Predeterminado del navegador

Por lo tanto, un estilo en línea tiene la prioridad más alta y anulará los estilos externos y estilos internos y valores predeterminados del navegador.

```
<!DOCTYPE html>
<html>

<head>
   <!-- Externo Prioridad 3 Color Azul -->
   <link rel="stylesheet" type="text/css" href="miestilo.css">
   <!-- Interno  Prioridad 2 Color: Coral-->
   <style>
      body {
         background-color: lightcoral;
      }
   </style>
</head>
<!-- En Línea  Prioridad 1 Color: Amarillo-->
<body style="background-color: lightyellow">
   <h1>Múltiples estilos se fusionarán en cascada en uno</h1>
   <p>Aquí, el color de fondo de la página se establece con
CSS en línea, y también con un CSS interno, y también con
      un CSS externo.</p>
   <p>Intente experimentar eliminando estilos para ver cómo
funcionan las hojas de estilo en cascada (intente eliminar
      primero el CSS en línea, luego el interno y luego el
externo).</p>

</body>

</html>
```

Comentarios en CSS

Los comentarios se utilizan para explicar su código, y pueden ser de ayuda el editar el código fuente. Los comentarios son ignorados por los navegadores.
Un comentario CSS comienza con **/*** y termina con ***/**. Los comentarios tabién pueden abarcar varias líneas.

Ejemplo

```
/*Selector de elemento, en este caso el elemento
es el Encabezdo h1*/
h1 {
    /*Declaraciones para aplicar estilos
    Declaración = propiedad : valor */
    font-family: Arial, Sans-serif; /*Seleccionamos como
fuente Arial*/
    text-align: center; /*Alineamos el texto en el centro*/

    color: #0000FF; /*Aplicamos el color*/
    font-size: 19pt; /*Aplicamos Tamaño de la fuente*/
}
```

Selectores en CSS

Los selectores CSS se utilizan para "encontrar" (o seleccionar) los elementos HTML a los que queremos dar estilo.

Podemos dividir los selectores CSS en cinco categorías:

1) Selectores simples (seleccione elementos según el nombre, la identificación, la clase)
2) Selectores de combinación (seleccione elementos basados en una relación específica entre ellos)
3) Selectores de Pseudo-Clase (seleccionar elementos en función de un estado determinado)
4) Selectores de Pseudo-Elementos (selecciona y aplica estilo a una parte de un elemento)
5) Selectores de atributos (seleccione elementos en función de un atributo o valor de atributo)

Selectores Simples

Los selectores simples en CSS permiten seleccionar elementos de una página web para aplicarles estilos. Son fáciles de usar y forman la base de muchos estilos en CSS.

Existen varios tipos de selectores simples:

1) Selector de Tipo o Etiqueta:
2) Selector de Clase
3) Selector de ID
4) Selector Universal
5) Selector de Grupo

Selector	Descripción
*	Selecciona todos los elementos del DOM
etiqueta	Selecciona todas las etiquetas indicadas

4.1 Selectores Simples.

1) Selector de Tipo o Etiqueta

Selecciona todos los elementos de un tipo específico (por su etiqueta HTML).
Ejemplo: este selector aplicará el color azul a todos los elementos <h1> de la página.

```
h1 {
    color: blue;
}
```

```
<h1> Noticia 1 con estilo aplicado por selector de equita h1</h1>
<h1> Noticia 2 con estilo aplicado por selector de equita h1</h1>
<h2>Comentarios no le afecta por que es un encabezado h2</h2>
```

Los selectores de tipo tienen la ventaja de que, con un simple cambio en la Hoja de Estilo, podemos modificar el aspecto de *todos* los elementos de ese tipo en *todas* las páginas de nuestro sitio. Esa es también su limitación: su utilidad se limita al caso en que *todos* los elementos de un tipo tengan la misma apariencia en cada una de nuestras páginas.

¿Qué sucede si en ciertos casos queremos usar un párrafo con características diferenciadas?.

2) Selector de Clase (.)

Solucionemos la cuestión anteriormente planteada: se puede conseguir **que haya más de un estilo para cada etiqueta HTML** utilizando las clases. El selector de clases consta de un punto (.) seguido por el nombre de la clase que hemos creado.

Para seleccionar los elementos con una clase específica escrebimos un carácter de punto (.) seguido del nombre de la clase

En el siguiente ejemplo, todos los elementos HTML con class="center" serán alieneados en el centro y con color de texto rojo.

Hoja de Estilo CSS

```
.centrado {
    text-align: center;
}
```

Documento HTML

```
<h1 class="centrado">Encabezado Centrado</h1>
<p class="centrado">Párrafo Centrado</p>
<p>Párrafo no Afectado</p>
```

2) Selector de Clase (.)

También pueden especificarse que sólo un elemento HTML en concreto se vea afectado por una **class**.

En el siguiente ejemplo, todos los elementos <p> con class="centrado" serán alineados en el centro

Hoja de Estilo CSS

```
p.centrado {
    text-align: center;
    color: red;
}
```

Documnto HTML

```
<h1 class="centrado">Encabezado no afectado</h1>
<p class="centrado">Párrafo centrado en color rojo</p>
```

Ejemplo con Selectores de Tipo o Etiqueta y Selectores de Clase

Hoja de Estilo CSS misestilos.css

```
/*Selectores de Tipo o Etiqueta*/
h1 {
    color: red;
}

p {
    color: blue;
}

/*Selectores de Clase*/
p.invertido {
    color: white;
    background-color: black;
}

.colorido {
    color: green;
    background-color: yellow;
}
```

Documento HTML index.html

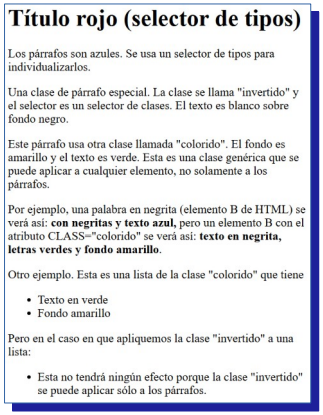
```
<html lang="es">

<head>
    <title>Ejemplo de estilos</title>
    <link rel="stylesheet" href="misestilos.css">
</head>

<body>
    <h1>Título rojo (selector de tipos)</h1>
    <p>Los párrafos son azules. Se usa un selector de tipos para individualizarlos.</p>
    <p class="invertido">Una clase de párrafo especial. La clase se llama "invertido" y el selector es un selector de clases.
        El texto es blanco sobre fondo negro.
    </p>
    <p class="colorido">Este párrafo usa otra clase llamada "colorido".   El fondo es amarillo y el texto es verde.
        Esta es una clase genérica que se puede aplicar a cualquier elemento, no solamente a los párrafos.
    </p>
    <p>Por ejemplo, una palabra en negrita (elemento B de HTML) se verá así:
        <b>con negritas y texto azul,</b> pero un elemento B con el
        atributo CLASS="colorido" se verá así: <b class="colorido">texto
            en negrita, letras verdes y fondo amarillo</b>.
    </p>
    <p>Otro ejemplo. Esta es una lista de la clase "colorido" que tiene</p>
    <ul class="colorido">
        <li>Texto en verde</li>
        <li>Fondo amarillo</li>
    </ul>
    <p>Pero en el caso en que apliquemos la clase &quot;invertido&quot;
        a una lista:</p>
    <ul class="invertido">
        <li>Esta no tendrá ningún efecto porque la clase &quot;invertido&quot;
            se puede aplicar sólo a los párrafos.</li>
    </ul>
</body>

</html>
```

HTML



Título rojo (selector de tipos)

Los párrafos son azules. Se usa un selector de tipos para individualizarlos.

Una clase de párrafo especial. La clase se llama "invertido" y el selector es un selector de clases. El texto es blanco sobre fondo negro.

Este párrafo usa otra clase llamada "colorido". El fondo es amarillo y el texto es verde. Esta es una clase genérica que se puede aplicar a cualquier elemento, no solamente a los párrafos.

Por ejemplo, una palabra en negrita (elemento B de HTML) se verá así: **con negritas y texto azul,** pero un elemento B con el atributo CLASS="colorido" se verá así: **texto en negrita, letras verdes y fondo amarillo**.

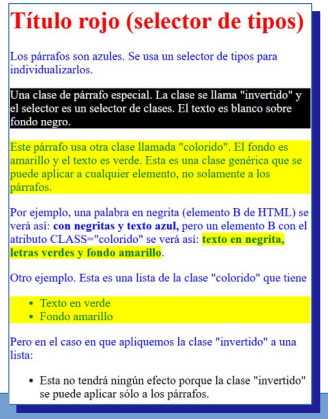
Otro ejemplo. Esta es una lista de la clase "colorido" que tiene

- Texto en verde
- Fondo amarillo

Pero en el caso en que apliquemos la clase "invertido" a una lista:

- Esta no tendrá ningún efecto porque la clase "invertido" se puede aplicar sólo a los párrafos.

HTML +CSS



Título rojo (selector de tipos)

Los párrafos son azules. Se usa un selector de tipos para individualizarlos.

Una clase de párrafo especial. La clase se llama "invertido" y el selector es un selector de clases. El texto es blanco sobre fondo negro.

Este párrafo usa otra clase llamada "colorido". El fondo es amarillo y el texto es verde. Esta es una clase genérica que se puede aplicar a cualquier elemento, no solamente a los párrafos.

Por ejemplo, una palabra en negrita (elemento B de HTML) se verá así: **con negritas y texto azul,** pero un elemento B con el atributo CLASS="colorido" se verá así: **texto en negrita, letras verdes y fondo amarillo**.

Otro ejemplo. Esta es una lista de la clase "colorido" que tiene

- Texto en verde
- Fondo amarillo

Pero en el caso en que apliquemos la clase "invertido" a una lista:

- Esta no tendrá ningún efecto porque la clase "invertido" se puede aplicar sólo a los párrafos.

3) Selector de ID

El selector ID utiliza el atributo id de un elemento HTML para seleccionar un elemento específico.

El id de un elemento es único dentro de una página, por lo que **el selector id se utiliza si se quiere seleccionar un ELEMENTO ÚNICO.**

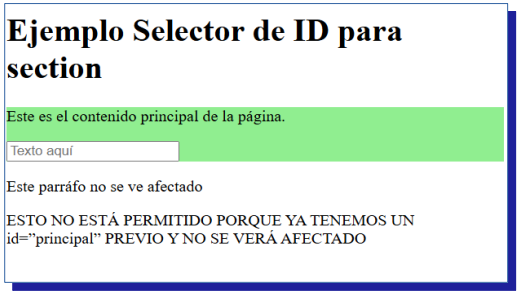
Para seleccionar un elemento con un identificador específico, escriba un carácter de almohadilla (#), seguido del ID del elemento.

Hoja de Estilo CSS

```
#principal {
    background-color: lightgray;
}
```

Documnto HTML

```
<h1>Ejemplo Selector de ID para section</h1>
    <section id="principal">
        <p>Este es el contenido principal de la página.</p>
        <input type="text" placeholder="Texto aquí">
    </section>
    <p>Este parráfo no se ve afectado</p>
        <p  id=”principal”>ESTO  NO  ESTÁ  PERMITIDO  PORQUE  YA
TENEMOS UN id=”principal” PREVIO Y NO SE APLICARÁ
    </p>
```



4) Selector Universal

El selector universal (*) selecciona todo el código HTML en la página.

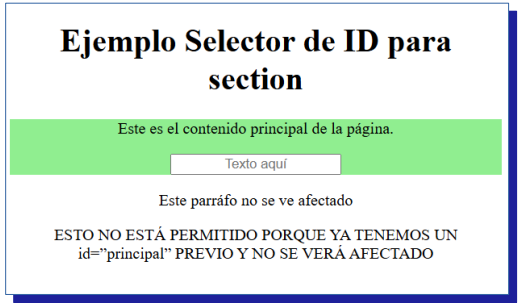
La siguiente regla CSS afectará a todos los elementos HTML de la página:

Hoja de Estilo CSS

```
* {
    text-align: center;
    color: red;
}
```

Documnto HTML

```
<h1>Ejemplo Selector de ID para section</h1>
    <section id="principal">
        <p>Este es el contenido principal de la página.</p>
        <input type="text" placeholder="Texto aquí">
    </section>
    <p>Este parráfo no se ve afectado</p>
        <p  id=”principal”>ESTO  NO  ESTÁ  PERMITIDO  PORQUE  YA
TENEMOS UN id=”principal” PREVIO Y NO SE APLICARÁ
    </p>
```



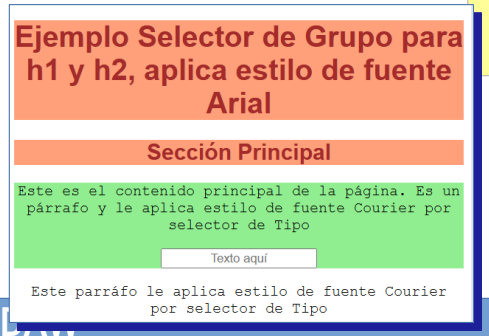
5) Selector de Grupo

En el caso de querer aplicar la misma regla para diferentes etiquetas podemos agruparlas de la siguiente forma

Hoja de Estilo CSS sin y con agrupación

```
h1{
  font-family:Arial;
  color: brown;
background-color:
lightsalmon;
}
h2{
  font-family:Arial;
  color: brown;
background-color:
lightsalmon;
}
p {
 font-family:Courier;
}
```

```
h1, h2{
  font-family:Arial;
  color: brown;
background-color:
lightsalmon;
}
p {
 font-family:Courier;
}
```



Documnto HTML

```
<h1>Ejemplo Selector de ID para section</h1>
<h2>Sección Principal</h2>
    <section id="principal">
        <p>Este es el contenido principal de la página.</p>
        <input type="text" placeholder="Texto aquí">
    </section>
    <p>Este parráfo no se ve afectado</p>
    <p id=”principal”>ESTO NO ESTÁ PERMITIDO PORQUE YA
TENEMOS UN id=”principal” PREVIO Y NO SE APLICARÁ
    </p>
```

Selectores de Combinación

Los selectores de combinación se llaman así porqué combinan otros selectores de manera que proporciona una relación útil entre ellos y la ubicación del contenido en el documento. Un selector CSS puede contener más de un selector simple, para ello entre lo selectores simples, podemos incluir un combinador.

1) Combinador de Descendiente (espacio)
2) Combinador Secundario o de Hijo Directo (>)
3) Combinador de Hermano Adyacente (+)
4) Combinador de Hermano General (~)

Selector	Descripción
sel1 sel2	Selección de los selectores sel2 que se encuentren dentro de los selectores sel1
.class1.class2	Selección de los elementos con las dos clases: class1 y class2
sel1.class1	Selección de todos los selectores sel1 con clase class1
sel1, sel2	Selección de todos los selectores separados por comas
sel1 > sel2	Selección de los selectores sel2 cuando son hijos de sel1
sel1 + sel2	Selección del selector sel2 cuando es hermano de sel1 (su elemento padre es el mismo)

Selectores de Combinación

Los selectores de combinación se llaman así porqué combinan otros selectores de manera que proporciona una relación útil entre ellos y la ubicación del contenido en el documento. Un selector CSS puede contener más de un selector simple, para ello entre lo selectores simples, podemos incluir un combinador.

1) Combinador de Descendiente (espacio)
2) Combinador Secundario o de Hijo Directo (>)
3) Combinador de Hermano Adyacente (+)
4) Combinador de Hermano General (~)

4.2 Selectores de Combinación.

1) Selector Combinador de Descenciente (espacio)

Estructura (elementoA elementoB)
Selecciona a todos los elementos B que están dentro del elemento A

```
/*Pone el texto centrado y en azul de todos los h2 de un section*/

section h2{
    color: blue;
    text-align: center;
}
```

Sección de Noticias

Noticia 1

Contenido Noticia 1

El año pasado se produjo.....

Noticia 3

Noticia 3

Sección de Reportajes

Reportaje 1

Reportaje 2

Reportaje 3

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
      <meta  name="viewport"  content="width=device-width,
initial-scale=1.0">
   <title>Document</title>
   <style>
      /* Pone el texto centrado en azul de
      todos los h2 dentro de un section */
      section h2{
         color: blue;
         text-align: center;
      }
   </style>
</head>
<body>
   Sección de Noticias
   <section>
      <h2>Noticia 1</h2>
      <article>
         <h2>Contenido Noticia 1</h2>
         <p>El año pasado se produjo.....</p>
      </article>
      <h2>Noticia 2</h2>
      <h2>Noticia 3</h2>
   </section>
   Sección de Reportajes
   <section>
      <h2>Reportaje 1</h2>
      <h2>Reportaje 2</h2>
      <h2>Reportaje 3</h2>
   </section>
</body>
</html>
```

2) Selector Combinador Secundario o de Hijo Directo (>)

Estructura (elementoA > elementoB)
Selecciona los elementos B que son hijos directos de A
En este caso el h2 que está dentro de article (no es hijo directo de section)
y no leafecta el estilo css

```
/*Pone el texto centrado y en azul de los h2 que sea hijos directos de un section*/

section > h2{
    color: blue;
    text-align: center;
}
```

Sección de Noticias

Noticia 1

Contenido Noticia 1

El año pasado se produjo.....

Noticia 2

Noticia 3

Sección de Reportajes

Reportaje 1

Reportaje 2

Reportaje 3

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
      <meta  name="viewport"  content="width=device-width, initial-scale=1.0">
   <title>Document</title>
   <style>
      /* Pone el texto centrado en azul de
      los h2 que sean hijos directos de un section */
      section > h2{
         color: blue;
         text-align: center;
      }
   </style>
</head>
<body>
   Sección de Noticias
   <section>
      <h2>Noticia 1</h2>
      <article>
         <h2>Contenido Noticia 1</h2>
         <p>El año pasado se produjo.....</p>
      </article>
      <h2>Noticia 2</h2>
      <h2>Noticia 3</h2>
   </section>
   Sección de Reportajes
   <section>
      <h2>Reportaje 1</h2>
      <h2>Reportaje 2</h2>
      <h2>Reportaje 3</h2>
   </section>
</body>
</html>
```

3) Selector Combinador de Hermano Adyacente (+)

Estructura (elementoA + elementoB)
Selecciona el elemento B que sigue inmediatamente después de A.
Ejemplo https://www.w3schools.com/css/css_combinators.asp

4) Selector Combinador de Hermano General (~)

Estructura (elementoA ~ elementoB)
Selecciona todos los elementos B que son hermanos de A dentro del mismo padre.
Ejemplo https://www.w3schools.com/css/css_combinators.asp

Selectores de Pseudo-Clase

Los selectores de Pseudo-Clases en CSS son palabras clave que puedes agregar a los selectores para aplicar estilos a un elemento en un estado particular, sin agregar una clase o ID especial.

Pseudo-clases para los estados de un elemento

Pseudo-clase	Descripción
:link	No visitado por el usuario
:visited	Visitado por el usuario
:hover	Modifica el estilo cuando un elemento apuntador pasa por encima
:active	Se activa cuando el usuario pulsa el elemento
:focus	Se activa cuando tiene el foco sobre el elemento

Enlace W3Schools

Pseudo-clases para selección de hijos o hermanos

Pseudo-clase	Descripción
:first-child	Primer hijo
:last-child	Último hijo
:first-of-type	Primer hermano de su tipo
:last-of-type	Último hermano de su tipo
:only-child	Hijos únicos
:only-of-type	Únicos hermanos de su tipo
:empty	Elementos que no tienen hijos
:nth-child(n)	Enésimo elemento hijo
:nth-last-child(n)	Enésimo elemento hijo contando desde el último
:nth-of-type(n)	Enésimo hermano de su tipo
:nth-last-of-type(n)	Enésimo hermano de su tipo comenzando desde el último

Selectores de Pseudo-Elementos

Enlace W3Schools

Un pseudoelemento CSS se utiliza para diseñar partes específicas de un elemento.
Por ejemplo, se puede utilizar para:

- Aplicar estilo a la primera letra o línea de un elemento
- Insertar contenido antes o después de un elemento
- Aplicar estilo a los marcadores de los elementos de la lista
- Aplicar estilo al cuadro de vista detrás de un cuadro de diálogo

Pseudo-clase	Descripción
::first-line	Primera línea de texto de un elemento
::first-letter	Primera letra de la primera línea de texto de un elemento
::before	Añade contenido al principio del documento
::after	Añade contenido al final del documento

Selectores de Atributos

Permiten aplicar estilo a los elementos HTML con atributos específicos

Es posible aplicar estilo a elementos HTML que tienen atributos específicos o valores de atributo.
Para más información usa el siguiente enlace, en el encontrarás ejemplos de su uso

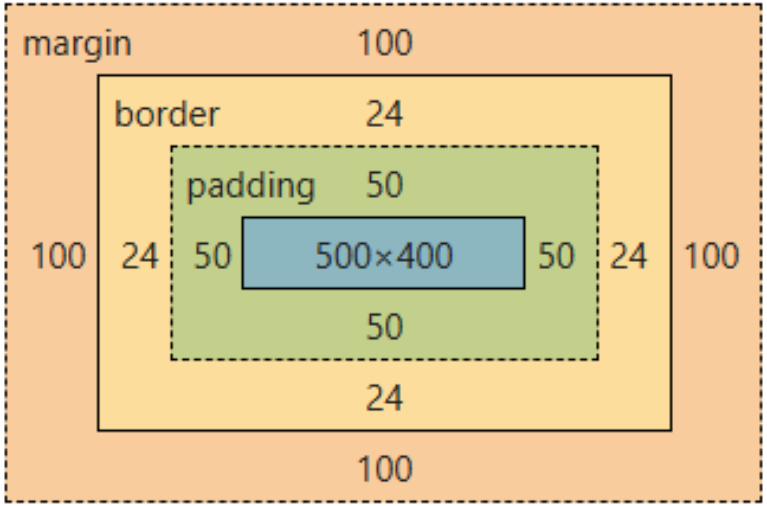
Enlace W3Schools

Cajas

En CSS, el modelo de cajas (o box model) es el principio fundamental que describe cómo se colocan y dimensionan los elementos en una página web. Cada elemento en HTML se representa como una caja rectangular en CSS, y esta caja tiene varias partes:

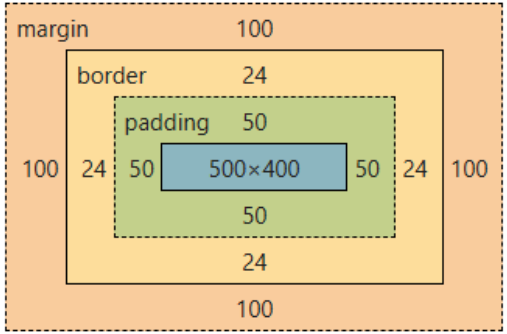
1) Content (Contenido): Es el área donde se muestra el contenido (texto, imágenes, etc.) del elemento.
2) Padding (Relleno): Es el espacio entre el contenido y el borde. Si el elemento tiene background-color, este color también se aplicará al área de padding.
3) Border (Borde): Es el borde alrededor del padding y el contenido. El grosor, estilo y color del borde se pueden personalizar con propiedades CSS como border-width, border-style, y border-color.
4) Margin (Margen): Es el espacio externo alrededor del borde. El margin no tiene color y separa la caja del elemento de otros elementos en la página.

Modelo de cajas (o box model)

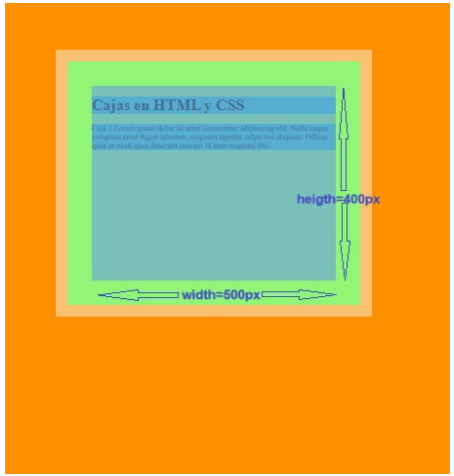


Ejemplo 1

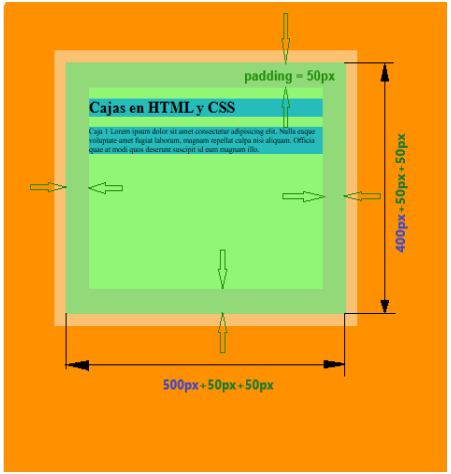
En el ejemplo suguiente tenemos una caja div a la que hemos dado unas dimensiones de 500px de ancho (width), 400px de alto (heigth), un padding de 50px, un borde de 50px y un margen de 100px, como podemos comprobar si abrimos el inspector en el modelo de cajas, que muestra la figura de la izquierda.



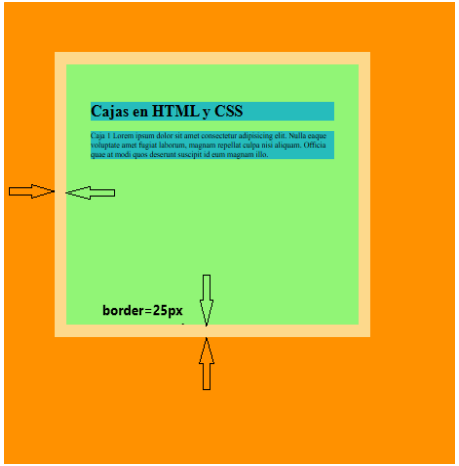
Contenido Ancho width = 500 px
Alto heigth = 400 px



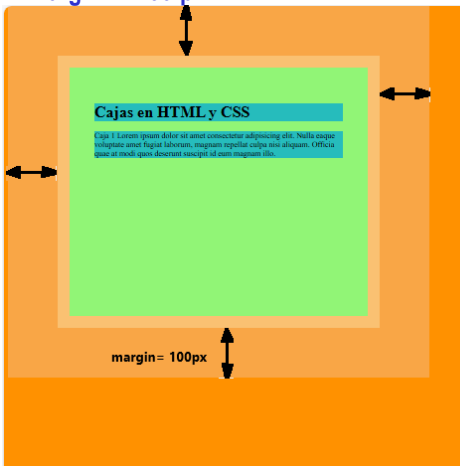
Relleno padding = 50 px



Borde padding = 25 px



Margen margin = 100 px



Layouts

Un layout es la estructura o disposición visual de los elementos en una página o pantalla. El layout define cómo se organizan y distribuyen los componentes como texto, imágenes, botones, menús, y otros elementos en el espacio disponible, para que sean visualmente coherentes, accesibles y funcionales para el usuario.

Tenemos cinco formar posibles de hacer Layouts o maquetado de páginas Web

1) **Tablas HTML (no recomendado):** el elemento table no se creo para usarlo para crear layouts. Aunque fue el primer método que se uso para hacer diseños web en los inicios. **No debemos usarlo**
2) **CSS float**: en una forma muy común de maquetar toda la web usando la propiedad de CSS float. Es fácil de aprender, pero en algunos casos no es todo lo flexible que necesitamos.
3) **CSS flexbox**: es un nuevo sistema de layout desde CSS3. Se va ajustando a las diferentes resoluciones de pantalla. Pero no funciona con Internet Explorer 10.
4) **CSS Grid Layout**: nos ofrece un layout basado en una rejilla con columnas y filas. Es fácil de posicionar los elementos. Pero no funciona con Internet Explorer 10 y Edge 15 o inferiores
5) **CSS framework**: es la forma más rápida de crear un layout, usando algún frameworks como lo es Bootstrap

Flujo normal

El flujo normal es el modo como el navegador presenta las páginas HTML de forma predeterminada cuando no haces nada para controlar el diseño de página, es decir, si no se aplica ningún CSS que cambie la forma en la que se comportan.

¿Cómo se presentan por defecto los elementos?

En primer lugar, se toma de las cajas de cada uno de los elementos el contenido, luego se añade cualquier área de relleno, borde y margen alrededor de ellas; es el modelo de cajas que hemos visto antes.

Tabla 1: Principales Tipos de Elementos en CSS

Tipo de Elemento	Descripción	Características	Ejemplos	Usos Comunes
Bloque (`block`)	Ocupan todo el ancho del contenedor y comienzan en una nueva línea.	- Ocupan todo el ancho disponible. - Permiten definir `width`, `height`, `margin`, `padding` en todas direcciones.	`<div>`, `<p>`, `<h1>`, `<ul>`, `<li>`	Estructura del contenido en secciones.
En línea (`inline`)	Ocupan solo el ancho del contenido y no generan nueva línea.	- Solo ocupan el ancho necesario. - No permiten `width` ni `height`, solo `padding` y `margin` horizontales.	`<span>`, `<a>`, `<strong>`, `<img>`	Estilos en fragmentos de texto.
En línea en bloque (`inline-block`)	Combina características de bloque e inline: ocupa solo el ancho necesario y permite dimensiones específicas.	- Acepta `width`, `height`, `padding` y `margin` en todas direcciones. - No ocupa toda la línea, permite elementos al lado.	`<button>`, `<input>`	Botones, íconos, elementos alineados.

Tabla 2: Elementos Especiales en CSS

Tipo de Elemento	Descripción	Características	Ejemplos	Usos Comunes
Flexible (`flex`)	Permite crear diseños adaptativos y flexibles en contenedores configurados como `display: flex`.	- Facilita alineación y distribución de elementos hijos horizontal o verticalmente. - Ideal para diseños responsivos.	Contenedores `display: flex`	Diseño de interfaces, disposiciones adaptativas.
Cuadrícula (`grid`)	Organiza contenido en filas y columnas en contenedores `display: grid`.	- Control preciso de posición en una cuadrícula bidimensional. - Facilita el diseño de estructuras complejas con varias columnas y filas.	Contenedores `display: grid`	Diseño de páginas con estructuras complejas.
Elemento de lista (`list-item`)	Para listas con viñetas o números, usado con `display: list-item`.	- Incluyen viñeta o número en listas. - Pueden convertirse en `inline` o `inline-block` si es necesario.	`<li>`	Organización de elementos en listas.

Flujo normal Ejemplo 3

Enlace

En el siguiente ejemplo se puede ver como es el Flujo Normal,

HTML

```html
<h1>Flujo de los documentos básicos</h1>

<p>
  Soy un elemento básico de nivel de bloque. Mis elementos de nivel de bloque
  adyacentes se encuentran en líneas nuevas debajo de mí.
</p>

<p>
  Cubrimos por defecto el 100% del ancho de nuestro elemento principal, y somos
  tan altos como nuestro contenido secundario. Nuestro ancho y alto total es
  nuestro contenido + área de relleno + ancho/alto del borde.
</p>

<p>
  Estamos separados por nuestros márgenes. Debido al colapso del margen, estamos
  separados por el ancho de uno de nuestros márgenes, no por ambos.
</p>

<p>
  Los elementos en línea <span>como este</span> y <span>este otro</span> se
  ubican en la misma y la de los nodos de texto adyacentes, mientras hay espacio
  en la misma línea. Si un elemento en línea desborda,
  <span
    >sigue por la línea siguiente, si es posible (como la que contiene este
    texto)</span
  >, o simplemente pasa a una línea nueva, como hace esta imagen:
  <img src="long.jpg" />
</p>
```

CSS

```css
body {
  width: 500px;
  margin: 0 auto;
}

p {
  background: rgba(255, 84, 104, 0.3);
  border: 2px solid rgb(255, 84, 104);
  padding: 10px;
  margin: 10px;
}

span {
  background: white;
  border: 1px solid black;
}
```

Flujo de los documentos básicos

Soy un elemento básico de nivel de bloque. Mis elementos de nivel de bloque adyacentes se encuentran en líneas nuevas debajo de mí.

Cubrimos por defecto el 100% del ancho de nuestro elemento principal, y somos tan altos como nuestro contenido secundario. Nuestro ancho y alto total es nuestro contenido + área de relleno + ancho/alto del borde.

Estamos separados por nuestros márgenes. Debido al colapso del margen, estamos separados por el ancho de uno de nuestros márgenes, no por ambos.

Los elementos en línea como este y este otro se ubican en la misma y la de los nodos de texto adyacentes, mientras hay espacio en la misma línea. Si un elemento en línea desborda, sigue por la línea siguiente, si es posible (como la que contiene este texto), o simplemente pasa a una línea nueva, como hace esta imagen:

Float

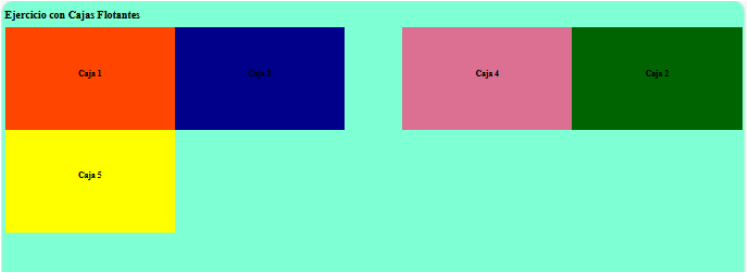
La propiedad **float** en CSS permite que un elemento "flote" a la izquierda o a la derecha dentro de su contenedor, haciendo que el texto y otros elementos se ajusten a su alrededor. Es útil para crear diseños donde se desea colocar elementos uno al lado del otro, pero puede requerir algunos ajustes para evitar problemas en el diseño.

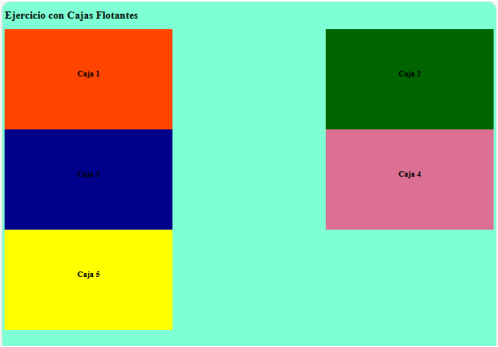
Valores de float:

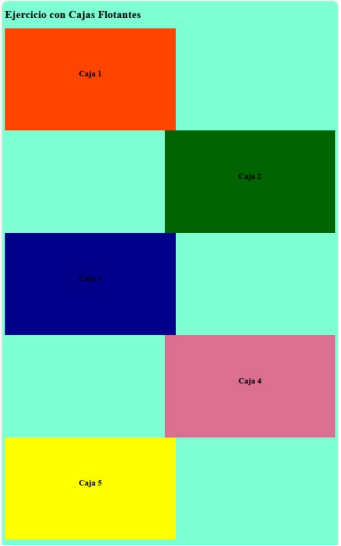
- **left**: El elemento flota a la izquierda de su contenedor
- **right**:-El elemento flota a la derecha de su contenedor
- **none**: El elemento no flota (se mostrará justo donde aparece en el texto). Este es el valor predeterminado
- **inherit**: El elemento hereda el valor float de su elemento primario

Ejemplo 2

En este ejemplo creamos 5 cajas con div de dimensiones 300x300px que floten las cajas impares a la izquierda y la pares a la derecha. El resultado es el que se muestra cuando cambiamos el tamaño de la pantalla que muestra nuestra página web







EJEMPLO 3 LAYOUT CON FLOATS

PASOS:

1.- Crea un archivo index.html con enlace a hoja estilos.css externa. En el html crea cuatro contenedores div a los que aplicamos estilo con la con clase box. En el css, crea la clase box para que tengan ancho y alto de 300px color gris, haz que floten a la ixquierda y pare separar las cajas dales un margen de 10px.

2.- Ahora vamos a modificar la primera caja para que tenga el aspecto de un header, para ello creamos la clase .header que hacemos que ocupe el 100% del ancho de nuestra página, flotante a la izquierda, con altura 70px, color salmon, texto alineado en el centro

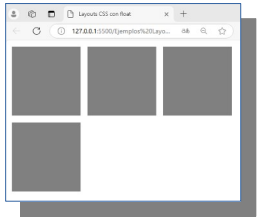
3.- Modificamos ahora la segunda caja para que sea una barra lateral que ocupe el 25% del ancho de la página, flotante a la izquierda, con altura 700px, color amarillo, texto alineado en el centro

4.- Modificamos la tercera caja para que sea el contenido

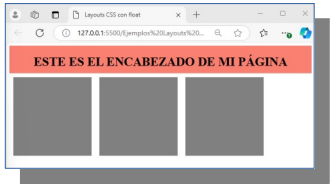
5.- Modificamos la última caja para que se el pie de página y quede con la apariencia final que se muestra

Como hemos usado para los anchos de las cajas porcentajes, cuando modificamos el tamaño de la pantalla se auto ajusta

Paso 1

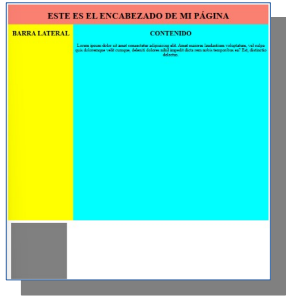


Paso 2

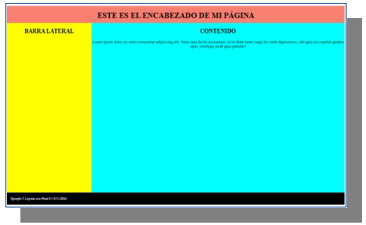


Paso 3

Paso 4



Paso 5



¿Qué es Flexbox?

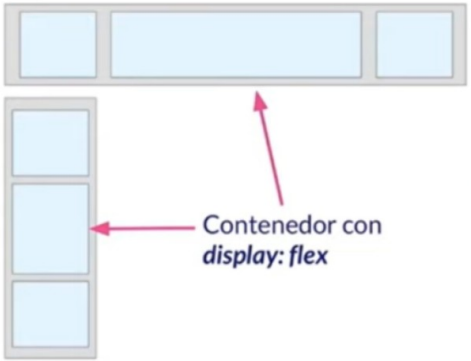
Flexbox es un método de diseño de página *unidimensional* para compaginar elementos en *filas o columnas* (por defecto está en filas). Los elementos HTML son flexibles, es decir, se adaptan y colocan automáticamente siendo mucho más fácil personalizar los diseños

¿Por qué flexbox?

Durante mucho tiempo, las únicas herramientas para crear diseños CSS fueron cosas como la flotación y el posicionamiento. Están bien y funcionan, pero limitan bastante y frustran.

Con tales herramientas resulta difícil, si no imposible, lograr obtener en cualquier forma conveniente y flexible un diseño de página sencillo con unos requisitos como los siguientes:

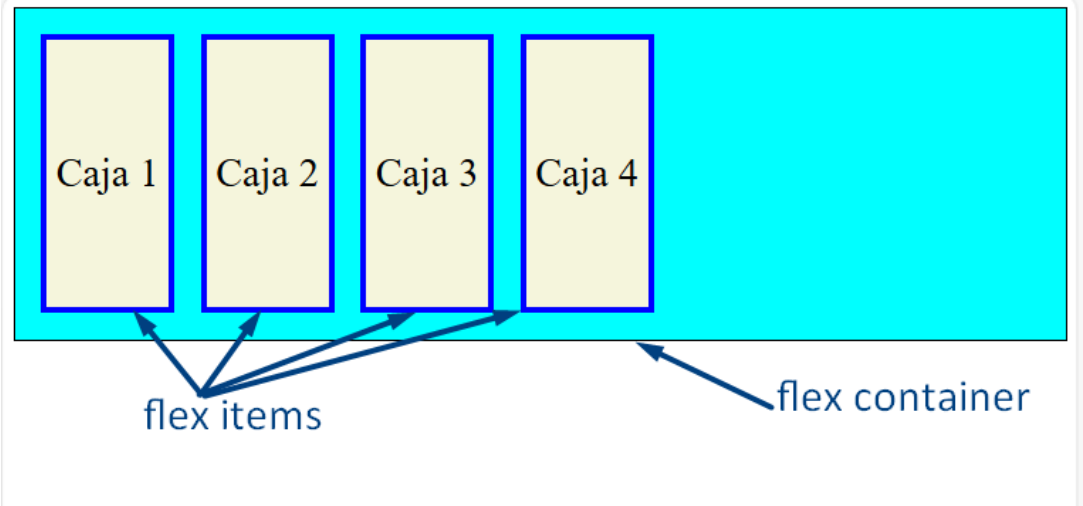
- Centrar verticalmente un bloque de contenido dentro de su elemento padre.
- Hacer que todos los elementos secundarios de un contenedor ocupen una cantidad igual del ancho/alto disponible, independientemente del ancho/alto que haya disponible.
- Hacer que todas las columnas en una compaginación en columnas múltiples adopten la misma altura incluso si contienen cantidades diferentes de contenido.



Los elementos flexbox facilitan mucho algunas tareas de compaginación.

En Flexbox tenemos...

- **Flexbox container:** es el contenedor que contiene los elementos flexibles.
- **Flexbox items:** son los elementos flexibles dentro del flex container




```html
<!-- CONTENEDOR PADRE DE LA CLASE flex-container -->
<div class="flex-container">
    <!-- Contenedores hijos de flex-container -->
    <div>Caja 1</div>
    <div>Caja 2</div>
    <div>Caja 3</div>
    <div>Caja 4</div>
</div>
```

```css
.flex-container{
    display: flex;
}
```

Propiedades del elemento padre (display: flex;)

Tenemos varias propiedades que podemos poner en nuestro flex-container o elemento padre (Nota: en cada uno de ellos si tiene un valor *predeterminado*)

- **flexbox-direction:** dirección en que distribuyen los elementos dentro del container, puede tomar los valores (column, column-reverse, *row* y row-reverse)
- **flexbox-wrap:** especifica si los elementos puede saltar a la siguiente fila/columna si es necesario, puede tomar los valores (wrap, *nowrap*, wrap-reverse)
- **flexbox-flow:** es una combinación de flexbox-direction y flexbox-wrap, nos permite poner en una sola declaración los dos valores, ejemplo flexbox-flow: column wrap;
- **justify-content:** sirve para alinear el contenido, puede tomar los valores (*flex-start*, flex-end, center, space-between, space-between, space-around y space-evenly)
- **align-items:** Alinea los items a lo largo del eje cruzado (perpendicular al eje principal), puede tomar los valores (*stretch*, flex-start, flex-end, center y baseline)
- **align-content:** Controla la alineación de las líneas de elementos flexibles cuando hay múltiples líneas **(solo se aplica si flex-wrap: wrap),** puede tomar los valores (*stretch*, flex-start, flex-end, center y baseline)

6.2. Layout con FLEXBOX. Resumen

flex-direction: dirección en que distribuyen los elementos dentro del container, puede tomar los valores

Valor	Descripción
row	Establece la dirección del eje principal en horizontal.
row-reverse	Establece la dirección del eje principal en horizontal (invertido).
column	Establece la dirección del eje principal en vertical.
column-reverse	Establece la dirección del eje principal en vertical (invertido).

flex-wrap: especifica si los elementos puede saltar a la siguiente fila/columna si es necesario, puede tomar los valores

Valor	Descripción
nowrap	Establece los ítems en una sola línea (no permite que se desborde el contenedor).
wrap	Establece los ítems en modo multilínea (permite que se desborde el contenedor).
wrap-reverse	Establece los ítems en modo multilínea, pero en dirección inversa.

6.2. Layout con FLEXBOX. Resumen

justify-content: Sirve para *colocar los ítems* de un contenedor mediante una disposición concreta *a lo largo del eje principal*:

Valor	Descripción
flex-start	Agrupa los ítems al inicio del eje principal.
flex-end	Agrupa los ítems al final del eje principal.
center	Agrupa los ítems en el dentro del eje principal.
space-between	Distribuye los ítems con espacio entre ellos(el mismo), pero sin espacio al inicio o al final.
space-around	Distribuye los ítems con espacio uniforme alrededor de cada elemento.
space-evenly	Distribuye los ítems con espacio uniforme entre ellos y en los extremos.

align-items: Se encarga de *alinear los ítems en el eje secundario del contenedor*. Actúa sobre cada una de las líneas de un contenedor multilinea (no tiene efecto sobre contenedores de una sola línea). Los valores que puede tomar son los siguientes:

Valor	Descripción
flex-start	Alinea los ítems al principio del eje secundario.
flex-end	Alinea los ítems al final del eje secundario.
center	Alinea los ítems al centro del eje secundario.
stretch	Alinea los ítems estirándolos de modo que cubran desde el inicio hasta el final del contenedor.
baseline	Alinea los ítems en el contenedor según la base del contenido de los ítems del contenedor.

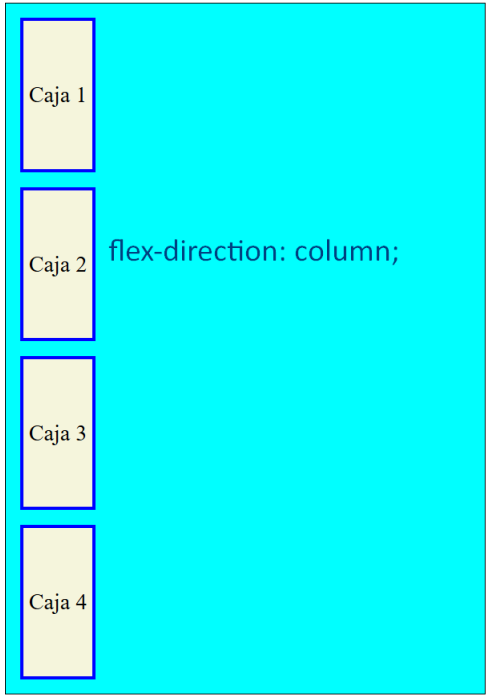
6.2. Layout con FLEXBOX. Resumen

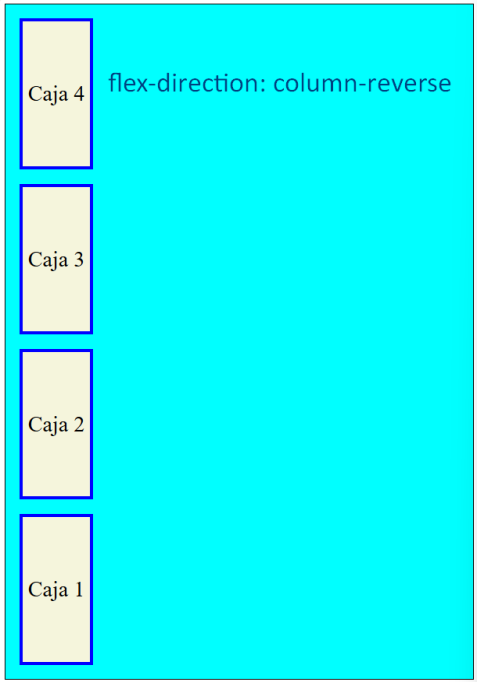
align-content: Controla la alineación de las líneas de elementos flexibles cuando hay múltiples líneas **(solo se aplica si flex-wrap: wrap),** puede tomar los valores (*stretch*, flex-start, flex-end, center y baseline)

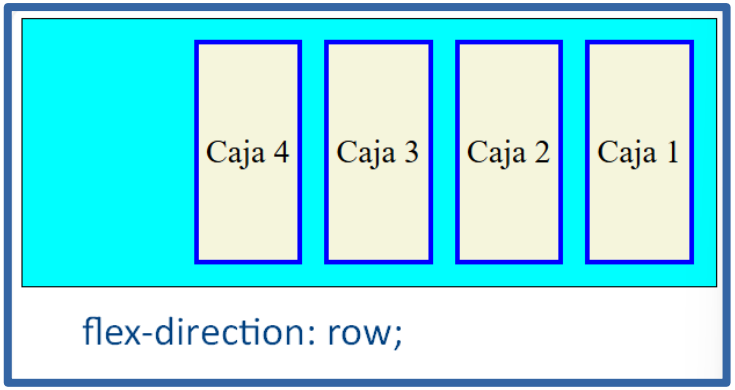
Valor	Descripción
flex-start	Agrupa los ítems al principio del eje principal.
flex-end	Agrupa los ítems al final del eje principal.
center	Agrupa los ítems al centro del eje principal.
space-between	Distribuye los ítems desde el inicio hasta el final.
space-around	Distribuye los ítems dejando el mismo espacio a los lados de cada uno.
stretch	Estira los ítems para ocupar de forma equitativa todo el espacio.

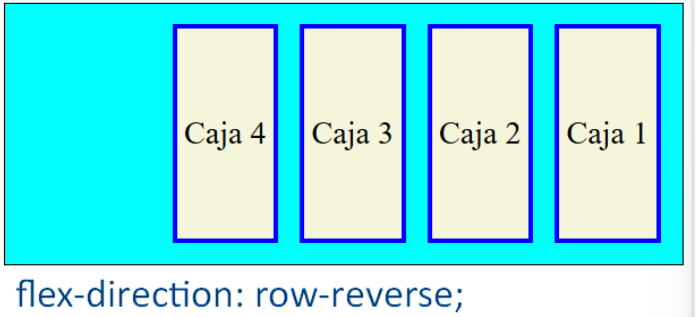
6.2. Layout con FLEXBOX

flexbox-direction: permiten modificar la dirección en que distribuyen los elementos dentro del container, puede tomar los valores (column, column-reverse, *row* y row-reverse)





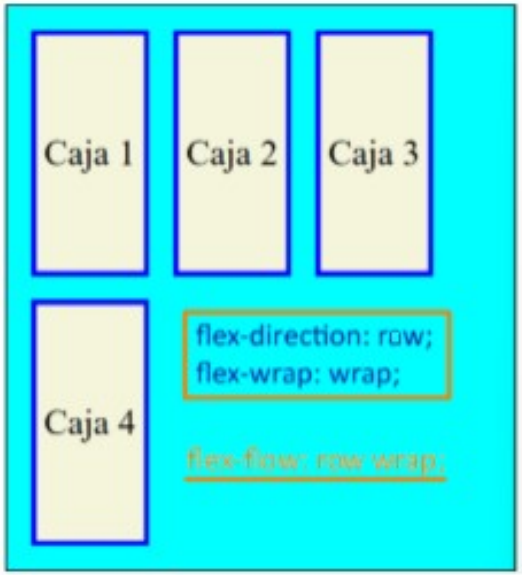


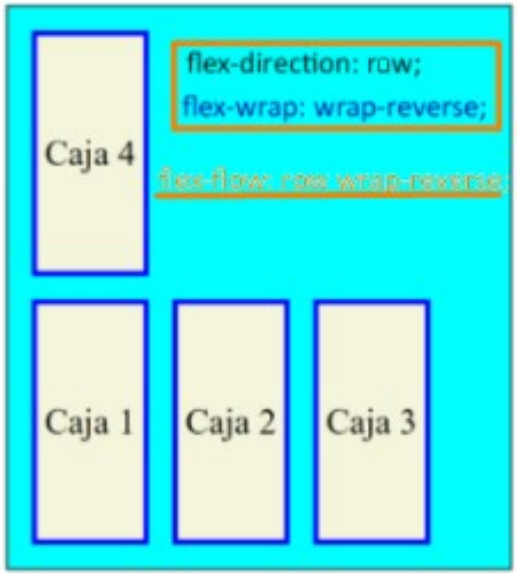


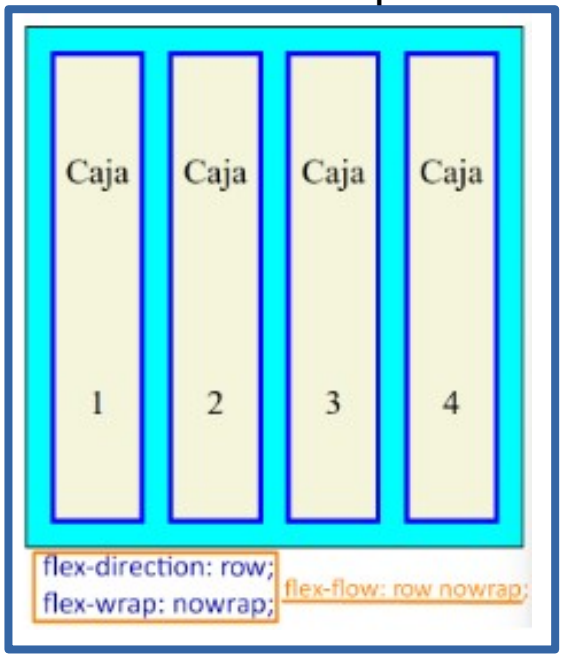
6.2. Layout con FLEXBOX

flexbox-wrap: especifica si los elementos puede saltar a la siguiente fila/columna si es necesario, puede tomar los valores (wrap, *nowrap*, wrap-reverse)

flexbox-flow: es una combinación de flexbox-direction y flexbox-wrap, nos permite poner en una sola declaración los dos valores, ejemplo flexbox-flow: row wrap-reverse;





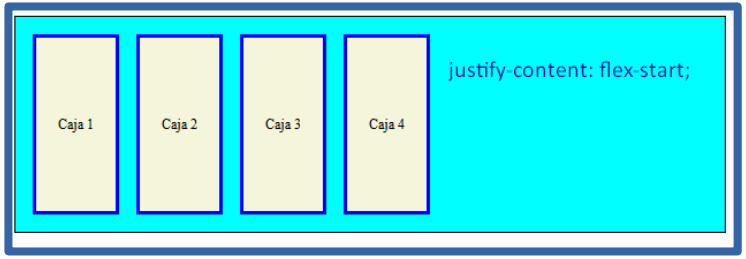


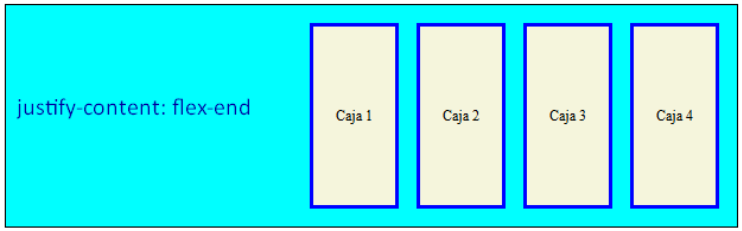
6.2. Layout con FLEXBOX

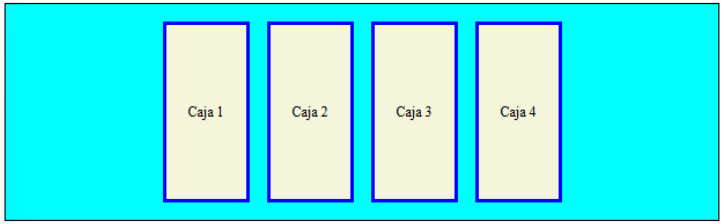
justify-content: sirve para alinear el contenido, por defecto a lo largo del eje horizontal

Puede tomar los valores:

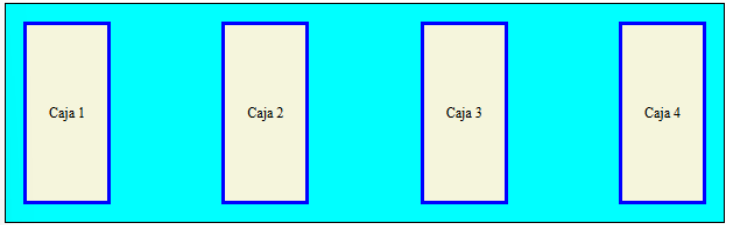
- *flex-start*
- flex-end,
- center
- space-between
- space-around
- space-evenly



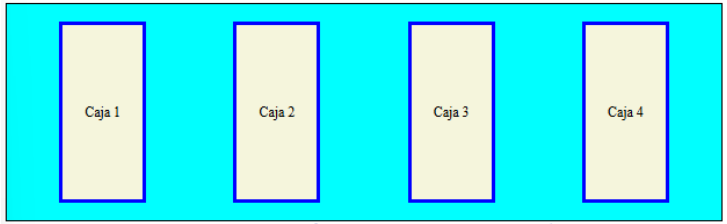




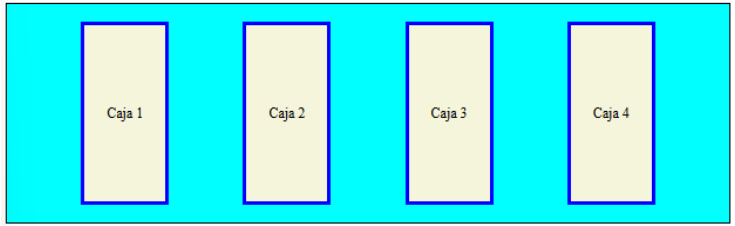
justify-content: center;



justify-content: space-between;



justify-content: space-around;



justify-content: space-evenly;

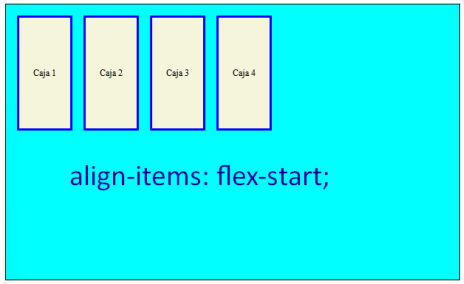
6.2. Layout con FLEXBOX

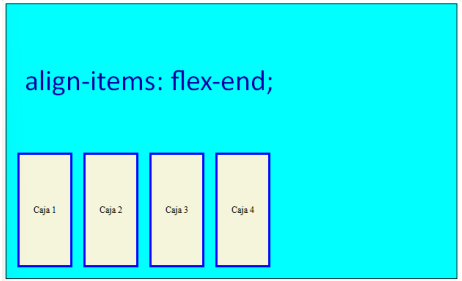
align-items: Se encarga de alinear los ítems en el eje secundario del contenedor. Actúa sobre cada una de las líneas de un contenedor multilinea (no tiene efecto sobre contenedores de una sola línea).

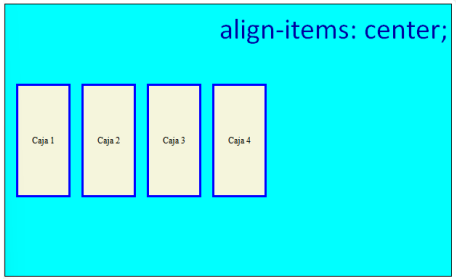
Para probarlo le damos altura al contenedor, ya que de otro modo se auto ajusta al contenido y no vemos como alinea verticalmente los items

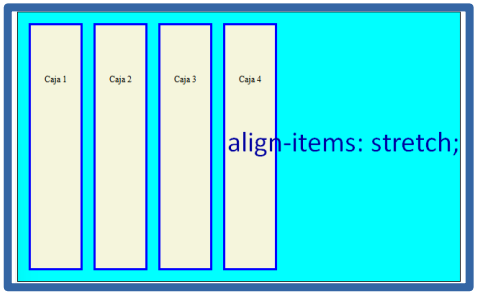
Puede tomar los valores:

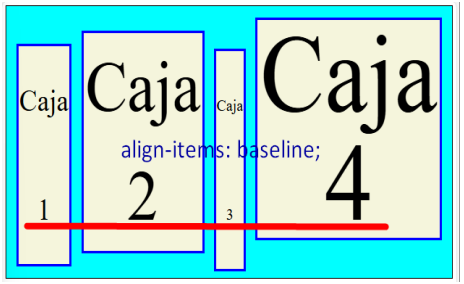
- flex-start
- flex-end,
- center
- *stretch*
- baseline









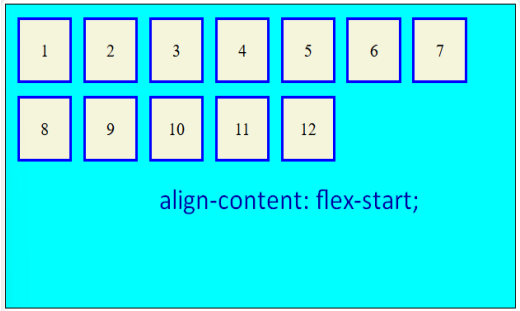


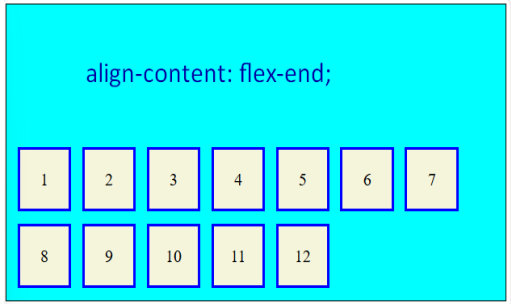
6.2. Layout con FLEXBOX

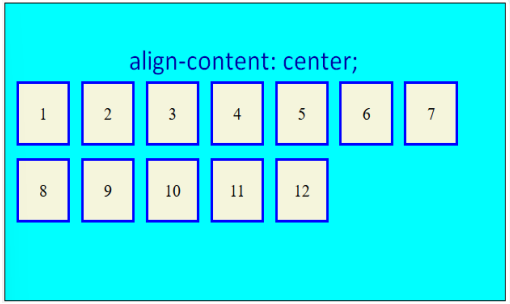
align-content: Controla la alineación de las líneas de elementos flexibles cuando hay múltiples líneas **(solo se aplica si flex-wrap: wrap),**

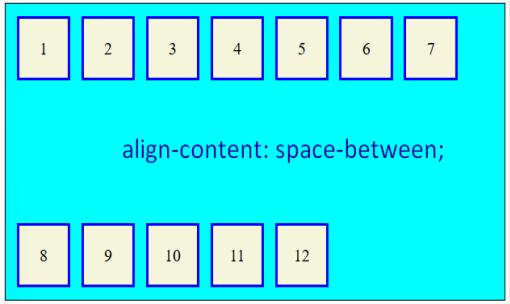
Puede tomar los valores:

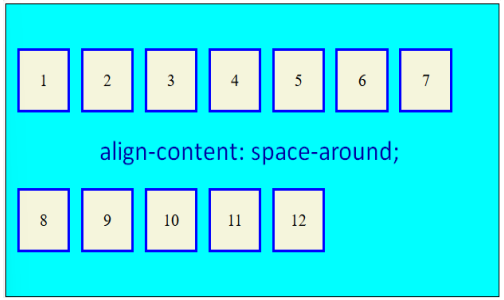
- flex-start
- flex-end,
- center
- space-between
- space-around
- *stretch*

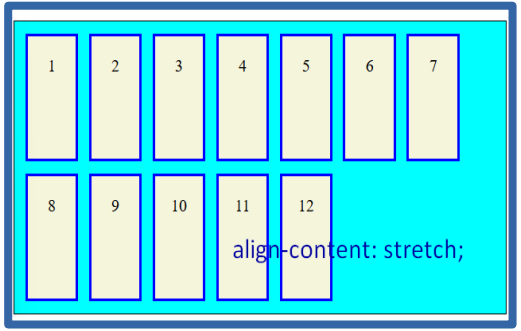












EJEMPLO 1 LAYOUT CON FLEXBOX

PASOS:

1.- Crea un archivo index.html con enlace a hoja estilos.css externa. En el html crea cuatro contenedores div a los que aplicamos estilo con la con clase box. En el css, crea la clase box para que tengan ancho y alto de 300px color gris, haz que floten a la ixquierda y pare separar las cajas dales un margen de 10px.

2.- Ahora vamos a modificar la primera caja para que tenga el aspecto de un header, para ello creamos la clase .header que hacemos que ocupe el 100% del ancho de nuestra página, flotante a la izquierda, con altura 70px, color salmon, texto alineado en el centro

3.- Modificamos ahora la segunda caja para que sea una barra lateral que ocupe el 25% del ancho de la página, flotante a la izquierda, con altura 700px, color amarillo, texto alineado en el centro

4.- Modificamos la tercera caja para que sea el contenido

5.- Modificamos la última caja para que se el pie de página y quede con la apariencia final que se muestra

Como hemos usado para los anchos de las cajas porcentajes, cuando modificamos el tamaño de la pantalla se auto ajusta